

Fire de execuție

Fire de execuție și sincronizare

O aplicație Java rulează în interiorul unui proces al sistemului de operare. Acest **proces** constă din segmente de cod și segmente de date mapate într-un spațiu virtual de adresare. Fiecare proces deține un număr de resurse alocate de către sistemul de operare, cum ar fi fișiere deschise, regiuni de memorie alocate dinamic, sau fire de execuție. Toate aceste resurse deținute de către un proces sunt eliberate la terminarea procesului de către sistemul de operare.

Un *fir de execuție* este unitatea de execuție a unui proces. Fiecare fir de execuție are asociate o secvență de instrucțiuni, un set de regiștri CPU și o stivă. Atenție, un proces nu execută nici un fel de instrucțiuni. El este de fapt un spațiu de adresare comun pentru unul sau mai multe fire de execuție. Execuția instrucțiunilor cade în responsabilitatea firelor de execuție. În cele ce urmează vom prescurta uneori denumirea firelor de execuție, numindu-le pur și simplu *fire*.

Mediul de execuție Java execută propriul său control asupra firelor de execuție. Algoritmul pentru planificarea firelor de execuție, prioritățile și stările în care se pot afla acestea sunt specifice aplicațiilor Java și implementate identic pe toate platformele pe care a fost portat mediul de execuție Java. Totuși, acest mediu știe să profite de resursele sistemului pe care lucrează. Dacă sistemul gazdă lucrează cu mai multe procesoare, Java va folosi toate aceste procesoare pentru a-și planifica firele de execuție.

Fire de execuție și sincronizare

În cazul mașinilor multiprocesor, mediul de execuție Java și sistemul de operare sunt responsabile cu repartizarea firelor de execuție pe un procesor sau altul. Pentru programator, acest mecanism este complet transparent, neexistând nici o diferență între scrierea unei aplicații cu mai multe fire pentru o mașină cu un singur procesor sau cu mai multe. Desigur, există însă diferențe în cazul scrierii aplicațiilor pe mai multe fire de execuție față de acelea cu un singur fir de execuție, diferențe care provin în principal din cauza necesității de sincronizare între firele de execuție aparținând aceluiași proces.

Sincronizarea firelor de execuție înseamnă că acestea se așteaptă unul pe celălalt pentru completarea anumitor operații care nu se pot executa în paralel sau care trebuie executate într-o anumită ordine. Java oferă și în acest caz mecanismele sale proprii de sincronizare, extrem de ușor de utilizat și înglobate în chiar sintaxa de bază a limbajului.

La lansarea în execuție a unei aplicații Java este creat automat și un prim fir de execuție, numit firul principal. Acesta poate ulterior să creeze alte fire de execuție care la rândul lor pot crea alte fire, și așa mai departe. Firele de execuție dintr-o aplicație Java pot fi grupate în grupuri pentru a fi manipulate în comun.

Fire de execuție și sincronizare

În afară de firele normale de execuție, Java oferă și fire de execuție cu prioritate mică care lucrează în fundalul aplicației atunci când nici un alt fir de execuție nu poate fi rulat. Aceste fire de fundal se numesc *demoni* și execută operații costisitoare în timp și independente de celelalte fire de execuție. De exemplu, în Java colectorul de gunoaie lucrează pe un fir de execuție separat, cu proprietăți de demon. În același fel poate fi gândit un fir de execuție care execută operații de încărcare a unor imagini din rețea.

O aplicație Java se termină atunci când se termină toate firele de execuție din interiorul ei sau când nu mai există decât fire demon. Terminarea firului principal de execuție nu duce la terminarea automată a aplicației.

Crearea firelor de execuție

Există două căi de definire de noi fire de execuție: derivarea din clasa *Thread* a noi clase și implementarea într-o clasă a interfeței *Runnable* .

În primul caz, noua clasă moștenește toate metodele și variabilele clasei *Thread* care implementează în mod standard, în Java, funcționalitatea de lucru cu fire de execuție.

Singurul lucru pe care trebuie să-l facă noua clasă este să reimplementeze metoda *run* care este apelată automat de către mediul de execuție la lansarea unui nou fir.

În plus, noua clasă ar putea avea nevoie să implementeze un constructor care să permită atribuirea unei denumiri firului de execuție.

Dacă firul are un nume, acesta poate fi obținut cu metoda *getName* care returnează un obiect de tip *String* .

Exemplu

```
public FirNou( String nume ) {  
    // apelează constructorul din Thread  
    super( nume );  
}  
  
public void run() {  
    while( true ) { // fără sfârșit  
        System.out.println( getName() +" Tastati ^C" );  
    }  
}  
}
```

```
public class TestFirNou {  
    public static void main( String[] arg) {  
        new FirNou( "Primul" ).start();  
    }  
}
```

Exemplu

```
public class PrimulFir {  
  
    public static void main(String arg[]) {  
  
        System.out.println("Crearea firelor de executie");  
  
        FirExecutie fir=new FirExecutie();  
  
        System.out.println("Pornim firul de executie");  
  
        fir.start();  
  
        System.out.println("revenire in main");  
  
    }  
}
```

```
class FirExecutie extends Thread {  
    public void run() {  
        int numar = 5;  
        System.out.println("Run se executa de "+numar +" ori");  
        for (int i=1;i<=numar;i++)  
            System.out.println("Pasul "+i);  
        System.out.println("Metoda un s-a incheiat");  
    }  
}
```

Metoda start, predefinită în obiectul Thread lansează execuția propriu-zisă a firului. Desigur există și căi de a opri execuția la nesfârșit a firului creat fie prin apelul metodei stop , prezentată mai jos, fie prin rescrierea funcției run în așa fel încât execuția sa să se termine după un interval finit de timp.

Creare prin implementare interfața Runnable

A doua cale de definiție a unui fir de execuție este implementarea interfeței Runnable într-o anumită clasă de obiecte.

Această cale este cea care trebuie aleasă atunci când clasa pe care o creăm nu se poate deriva din clasa Thread pentru că este important să fie derivată din altă clasă.

```
class Oclasa {
```

```
...
```

```
}
```

```
class FirNou extends Oclasa implements Runnable {
```

```
public void run() {
```

```
for( int i = 0; i < 100; i++ ) {
```

```
System.out.println( "pasul " + i );
```

```
}
```

```
}
```

```
...
```

```
}
```

```
public class TestFirNou {
```

```
public static void main( String argumente[ ] ) {
```

```
new Thread( new FirNou() ).start();
```

```
// Obiectele sunt create și folosite imediat
```

```
// La terminarea instrucțiunii, ele sunt automat
```

```
// eliberate nefiind referite de nimic
```

```
}
```


Stările unui fir de execuție

Un fir de execuție se poate afla în Java în mai multe stări, în funcție de ce se întâmplă cu el la un moment dat.

Atunci când este creat, dar înainte de apelul metodei `start`, firul se găsește într-o stare pe care o vom numi **Fir Nou Creat**. În această stare, singurele metode care se pot apela pentru firul de execuție sunt metodele `start` și `stop`.

Metoda `start` lansează firul în execuție prin apelul metodei `run`.

Metoda `stop` omoară firul de execuție încă înainte de a fi lansat. Orice altă metodă apelată în această stare provoacă terminarea firului de execuție prin generarea unei excepții de tip **`IllegalThreadStateException`**.

Dacă apelăm metoda `start` pentru un **Fir Nou Creat** firul de execuție va trece în starea **Rulează**. În această stare, instrucțiunile din corpul metodei `run` se execută una după alta. Execuția poate fi oprită temporar prin apelul metodei `sleep` care primește ca argument un număr de milisecunde care reprezintă intervalul de timp în care firul trebuie să fie oprit.

După trecerea intervalului, firul de execuție va porni din nou.

Stările unui fir de execuție

În timpul în care se scurge intervalul specificat de **sleep**, obiectul nu poate fi repornit prin metode obișnuite.

Singura cale de a ieși din această stare este aceea de a apela metoda **interrupt**. Această metodă generează o excepție de tip **InterruptedException** care nu este interceptată de **sleep** dar care trebuie interceptată obligatoriu de metoda care a apelat metoda **sleep**. De aceea, modul standard în care se apelează metoda **sleep** este următorul:

```
...
try {
    sleep( 1000 ); // o secundă
} catch( InterruptedException ) {
    ...
}
...
```

Stările unui fir de execuție

Dacă dorim oprirea firului de execuție pe timp nedefinit, putem apela metoda `suspend`. Aceasta trece firul de execuție într-o nouă stare, numită **Nu Rulează**. Aceeași stare este folosită și pentru oprirea temporară cu `sleep`. În cazul apelului `suspend` însă, execuția nu va putea fi reluată decât printr-un apel al metodei `resume`. După acest apel, firul va intra din nou în starea **Rulează**.

Pe timpul în care firul de execuție se găsește în starea **Nu Rulează**, acesta nu este planificat niciodată la controlul unității centrale, aceasta fiind cedată celorlalte fire de execuție din aplicație.

Firul de execuție poate intra în starea **Nu Rulează** și din alte motive. De exemplu se poate întâmpla ca firul să aștepte pentru terminarea unei operații de intrare/ieșire de lungă durată caz în care firul va intra din nou în starea **Rulează** doar după terminarea operației.

O altă cale de a ajunge în starea **Nu Rulează** este aceea de a apela o metodă sau o secvență de instrucțiuni sincronizată după un obiect. În acest caz, dacă obiectul este deja blocat, firul de execuție va fi oprit până în clipa în care obiectul cu pricina apelează metoda `notify` sau `notifyAll`.

Stările unui fir de execuție

Atunci când metoda **run** și-a terminat execuția, obiectul intră în starea **Mort**. Această stare este păstrată până în clipa în care obiectul este eliminat din memorie de mecanismul de colectare a gunoaielor. O altă posibilitate de a intra în starea **Mort** este aceea de a apela metoda **stop**.

Firul de execuție poate fi terminat și pe alte căi, caz în care metoda **stop** nu este apelată. În aceste situații este preferabil să ne folosim de o clauză **finally** ca în exemplul următor:

```
...
try {
    firDeExecutie.start( );
    ...
} finally {
    ..// curățenie
}
```

În fine, dacă nu se mai poate face nimic pentru că firul de execuție nu mai răspunde la comenzi, puteți apela la metoda **destroy**. Din păcate, metoda **destroy** termină firul de execuție fără a proceda la curățirile necesare în memorie.

Stările unui fir de execuție

Atunci când un fir de execuție este oprit cu comanda **stop**, mai este nevoie de un timp până când sistemul efectuează toate operațiile necesare opririi. Din această cauză, este preferabil să așteptăm în mod explicit terminarea firului prin apelul metodei **join**:

```
firDeExecutie.stop( )  
  
try {  
    firDeExecutie.join( );  
} catch( InterruptedException e ) {  
  
    ...  
}
```

Excepția de întrerupere trebuie interceptată obligatoriu. Dacă nu apelăm metoda **join** pentru a aștepta terminarea și metoda **stop** este de exemplu apelată pe ultima linie a funcției **main**, există șansa ca sistemul să creadă că firul auxiliar de execuție este încă în viață și aplicația Java să nu se mai termine rămânând într-o stare de așteptare. O puteți desigur termina tastând ^C.

Prioritatea firelor de execuție

Fiecare fir de execuție are o prioritate cuprinsă între valorile `MIN_PRIORITY` și `MAX_PRIORITY`. Aceste două variabile finale sunt declarate în clasa `Thread`. În mod normal însă, un fir de execuție are prioritatea `NORM_PRIORITY`, de asemenea definită în clasa `Thread`.

Mediul de execuție Java planifică firele de execuție la controlul unității centrale în funcție de prioritatea lor. Dacă există mai multe fire cu prioritate maximă, acestea sunt planificate după un algoritm numit round-robin. Firele de prioritate mai mică intră în calcul doar atunci când toate firele de prioritate mare sunt în starea **Nu Rulează**.

Prioritatea unui fir de execuție se poate interoga cu metoda `getPriority` care întoarce un număr întreg care reprezintă prioritatea curentă a firului de execuție. Pentru a seta prioritatea, se folosește metoda `setPriority` care primește ca parametru un număr întreg care reprezintă prioritatea dorită.

Există însă situații în care putem schimba această prioritate fără pericol, de exemplu când avem un fir de execuție care nu face altceva decât să citească caractere de la utilizator și să le memoreze într-o zonă temporară. În acest caz, firul de execuție este în cea mai mare parte a timpului în starea **Nu Rulează** din cauză că așteaptă terminarea unei operații de intrare/ieșire.

Prioritatea firelor de execuție

În alte situații, avem de executat o sarcină cu prioritate mică. În aceste cazuri, putem seta pentru firul de execuție care execută aceste sarcini o prioritate redusă.

Alternativ, putem defini firul respectiv de execuție ca un **demon**.

Dezavantajul în această situație este faptul că aplicația va fi terminată atunci când există doar demoni în lucru și există posibilitatea pierderii de date.

Pentru a declara un fir de execuție ca demon, putem apela metoda **setDaemon**. Această metodă primește ca parametru o valoare booleană care dacă este true firul este făcut demon și dacă nu este adus înapoi la starea normală.

Putem testa faptul că un fir de execuție este demon sau nu cu metoda **isDaemon**.

Grupuri de fire de execuție

Din acest motiv, este util să putem grupa firele de execuție pe grupuri. Această funcționalitate este oferită în Java de către o clasă numită **ThreadGroup**.

La pornirea unei aplicații Java, se creează automat un prim grup de fire de execuție, numit grupul principal, **main**. Firul principal de execuție face parte din acest grup. În continuare, ori de câte ori creăm un nou fir de execuție, acesta va face parte din același grup de fire de execuție ca și firul de execuție din interiorul căruia a fost creat, în afară de cazurile în care în constructorul firului specificăm explicit altceva.

Într-un grup de fire de execuție putem defini nu numai fire dar și alte grupuri de execuție. Se creează astfel o arborescență a cărei rădăcină este grupul principal de fire de execuție.

Pentru a specifica pentru un fir un nou grup de fire de execuție, putem apela constructorii obișnuiți dar introducând un prim parametru suplimentar de tip **ThreadGroup**.

Exemplu

```
ThreadGroup tg = new ThreadGroup( "Noul grup" );
```

```
Thread t = new Thread( tg, "Firul de executie" );
```

Acest nou fir de execuție va face parte dintr-un alt grup de fire decât firul principal. Putem afla grupul de fire de execuție din care face parte un anumit fir apelând metoda `getThreadGroup` , ca în secvența:

```
Thread t = new Thread( "Firul de Executie" );
```

```
ThreadGroup tg = t.getThreadGroup();
```

Operațiile definite pentru un grup de fire de execuție sunt clasificabile în operații care acționează la nivelul grupului, cum ar fi aflarea numelui, setarea unei priorități maxime, etc., și operații care acționează asupra fiecărui fir de execuție din grup, cum ar fi stop, suspend sau resume.

Enumerarea firelor de execuție

Pentru a enumera firele de execuție active la un moment dat, putem folosi metoda `enumerate` definită în clasa `Thread` precum și în clasa `ThreadGroup`.

Această metodă primește ca parametru o referință către un tablou de referințe la obiecte de tip `Thread` pe care îl umple cu referințe către fiecare fir activ în grupul specificat.

Pentru a afla câte fire active sunt în grupul respectiv la un moment dat, putem apela metoda `activeCount` din clasa `ThreadGroup`. De exemplu:

```
ThreadGroup grup = Thread.currentThread().getThreadGroup();  
int numarFire = grup.activeCount();  
Thread fire[] = new Thread[numarFire];  
grup.enumerate( fire );  
for( int i = 0; i < numar; i++ ) {  
    System.out.println( fire[i].toString() );  
}
```

Metoda `enumerate` întoarce numărul de fire memorate în tablou, care este identic cu numărul de fire active.

Sincronizare

În unele situații se poate întâmpla ca mai multe fire de execuție să vrea să acceseze aceeași variabilă. În astfel de situații, se pot produce încurcături dacă în timpul unuia dintre accese un alt fir de execuție modifică valoarea variabilei.

Limbajul Java oferă în mod nativ suport pentru protejarea acestor variabile. Putem defini metode, în cadrul claselor, care sunt sincronizate.

Pe o instanță de clasă, la un moment dat, poate lucra o singură metodă sincronizată. Dacă un alt fir de execuție încearcă să apeleze aceeași metodă pe aceeași instanță sau o altă metodă a clasei de asemenea declarată sincronizată, acest al doilea apel va trebui să aștepte înainte de execuție eliberarea instanței de către cealaltă metodă.

În afară de sincronizarea metodelor, se pot sincroniza și doar blocuri de instrucțiuni. Aceste sincronizări se fac tot în legătură cu o anumită instanță a unei clase. Aceste blocuri de instrucțiuni sincronizate se pot executa doar când instanța este liberă. Se poate întâmpla ca cele două tipuri de sincronizări să se amestece, în sensul că obiectul poate fi blocat de un bloc de instrucțiuni și toate metodele sincronizate să aștepte, sau invers.

Declararea unui bloc de instrucțiuni sincronizate se face prin:

```
synchronize ( Instanță ) {  
  
    Instrucțiuni  
  
}
```

iar declararea unei metode sincronizate se face prin folosirea modifierului `synchronize` la implementarea metodei.

Exemplu. Problema filozofilor

Exemplul următor implementează soluția următoarei probleme: Într-o țară foarte îndepărtată trăiau trei înțelepți filozofi. Acești trei înțelepți își pierdeau o mare parte din energie certându-se între ei pentru a afla care este cel mai înțelept. Pentru a tranșa problema o dată pentru totdeauna, cei trei înțelepți au pornit la drum către un al patrulea înțelept pe care cu toții îl recunoșteau că ar fi mai bun decât ei.

Când au ajuns la acesta, cei trei i-au cerut să le spună care dintre ei este cel mai înțelept. Acesta, a scos cinci pălării, trei negre și două albe, și li le-a arătat explicându-le că îi va lega la ochi și le va pune în cap câte o pălărie, cele două rămase ascunzându-le. După aceea, le va dezlega ochii, și fiecare dintre ei va vedea culoarea pălăriei celorlalți dar nu și-o va putea vedea pe a sa. Cel care își va da primul seama ce culoare are propria pălărie, acela va fi cel mai înțelept.

După explicație, înțeleptul i-a legat la ochi, le-a pus la fiecare câte o pălărie neagră și le-a ascuns pe celelalte două. Problema este aceea de a descoperi care a fost raționamentul celui care a ghicit primul că pălăria lui este neagră.

Exemplu. Problema filozofilor

Programul următor rezolvă problema dată în felul următor: Fiecarîntelept privește pălăriile celorlalți doi. Dacă ambele sunt albe, problema este rezolvată, a lui nu poate fi decât neagră. Dacă vede o pălărie albă și una neagră, atunci el va trebui să aștepte puțin să vadă ce spune cel cu pălăria neagră. Dacă acesta nu găsește soluția, înseamnă că el nu vede două pălării albe, altfel ar fi găsit imediat răspunsul. După un scurt timp de așteptare, înteleptul poate să fie sigur că pălăria lui este neagră.

În fine, dacă ambele pălării pe care le vede sunt negre, va trebui săaștepte un timp ceva mai lung pentru a vedea dacă unul dintre concurenții săi nu ghicește pălăria. Dacă după scurgerea timpului nici unul nu spune nimic, înseamnă că nici unul nu vede o pălărie albă și una neagră. Înseamnă că propria pălărie este neagră.

Desigur, raționamentul pleacă de la ideea că ne putem baza pe faptul că toți înteleptii gândesc și pot rezolva probleme ușoare. Cel care câștigă a gândit doar un pic mai repede. Putem simula viteza de gândire cu un interval aleator de așteptare până la luarea deciziilor. În realitate, intervalul nu este aleator ci dictat de viteza de gândire a fiecărui întelept.

Cei trei întelepți sunt implementați identic sub formă de fire de execuție. Nu câștigă la fiecare rulare același din cauza caracterului aleator al implementării. Înteleptul cel mare este firul de execuție principal care controlează activitatea celorlalte fire și le servește cu date, culoarea pălăriilor, doar în măsura în care aceste date trebuie să fie accesibile. Adică nu se poate cere propria culoare de pălărie.
